

95-产品数据条目化树形化及向后分配研究报告

版本：v0.5 | **修订：**2026-09-10 | **状态：**初稿（外部理论梳理，尚未与 EOS 实例逐条比对）

摘要：本报告回答"从相关方需求到构件的设计方案，链条上的产品数据能否**条目化**、**树形化**、**向后分配**，理论依据是什么"。结论是三者都成立，且在主流系统工程体系里不是可选项而是强制项；但三者都不是自然成立的——**每一条、每一层都要逐条判定**。三者的关系是一条链：**条目化提供最小单元 → 树形化提供组织 → 向后分配提供层间连接**。

取材限定：本报告只采用**外部权威**——国际标准、业界手册、公开发表的方法论与学术文献；不以本项目既有文档作为论据。§六 另述其对本项目的意味（属应用，不作为论据）。

核查说明：本机网络策略拦截了对 INCOSE、SEBoK、MIT 等站点的正文抓取，故引用来自搜索结果摘要与标准通行内容，未逐字核对原文。**29148 的九条特性措辞、15288 的过程编号建议依标准原文复核**（见 §七）。

术语（本报告反复用到的几个词）

词	含义
条目	有身份、可独立解释与验证的最小单位
叶 / 末级	一份产品数据的树中最深的那一级；但"最深"不等于"够格"——须切到 可被独立解释、独立验证、可被唯一交出去 的粒度（判据见 §4.5）
枝梢	概要态（尚未产出叶）的树中 最深 的那一级节点；在完整态中即"叶的上一级"
分配	把前序产品数据的叶，交给后续产品数据的节点（本报告回答的正是"交给谁、几对几"）
需求侧 / 方案侧	一对"需求—方案"中的两侧；前序产品数据是需求侧，后续产品数据是方案侧
FR / DP	公理化设计的两个基本概念：功能需求（FR）与设计参数（DP）
system element	标准用语，指"可被当作一个整体处理的系统成员"——分配的目标对象

一、问题与结论

1.1 本报告回答什么

从**相关方需求到构件的设计方案**，设计链条上的每一份产品数据，是否可以——

1. **条目化**：拆成有身份、可独立解释与验证的条目？

3. **向后分配**：由上一层逐条分配到下一层，并逐层向后推进？

若能，理论依据是什么。

1.2 结论一览

问题	结论	主要依据	本报告节
条目化	可以，标准强制	ISO/IEC/IEEE 29148 「良构需求」特性 (Singular / Unambiguous / Verifiable ...)	§二
树形化	可以，但以 三个条件 为前提 (判据单维 / 条目单归属 / 分组完备互斥)	Simon 层级与近可分解性；WBS 100% 规则 + MECE；INCOSE 规格树 ； 树序 ⇔ 递归划分 ；Bliss 「一次只用一个划分特征」；单继承与多层级 (DAG) 之分	§三
向后分配	可以；其 形状 为「叶 → 唯一枝梢、基数 N:1」，且两侧理由 不同 (N:1 靠内聚、1:N 靠耦合被禁)	ISO/IEC/IEEE 15288 技术过程链与 allocation；SEBoK；公理化设计 FR⇔DP zigzag ；Parnas 信息隐藏 与模块化；追溯性要求	§四

1.3 三问的关系：单元 → 组织 → 连接

三问不是并列的三件事，而是一层比一层外扩的三个问题，合起来才构成一条可推进的设计链：

- ① 条目化 — 一份产品数据内部，最小单位是什么？（规范论：一条长什么样）
↓ 提供可分配的最小单位（**Singular / Verifiable**）

② 树形化 — 一份产品数据内部，这些单位如何组织？（本体论：多条怎么摆）
↓ 提供可承接的层级与末级节点（层级 / **100%** 规则）

③ 向后分配 — 两份产品数据之间，如何连接？（方法论：层间怎么走）

↑ ↓

└——— 需要更细的条目 ←—— 方案侧架构“导出”下一层需求 ——┐

闭环之处在最后一环：分配不是终点——下一层的方案架构会**导出**更细的需求条目，成为再下一层的输入。这就是“逐层推进”之所以成立的机制。

1.4 各章的展开方式

第二章至第四章，每章都按同一顺序展开，便于对照阅读：

1. **依据**——这件事为什么成立（外部权威怎么说）；
2. **条件 / 形状**——它在什么条件下成立、具体是什么形状；
3. **小结**——本章的正面结论。

限度（反面边界）不放在各章，统一收在 §五，避免重复。

二、条目化：产品数据的最小单元

本问位置：链条的**最小单位**从哪来。**依据领域：**需求书写标准——ISO/IEC/IEEE 29148。

2.1 依据：良构需求的九条特性

ISO/IEC/IEEE 29148:2018 为**单条需求**定义了"良构 (well-formed)"的特性清单（九条，含）：

特性	含义（通行表述）
Singular	陈述只聚焦 单一 能力、特性或约束
Unambiguous	只能有 一种 解释
Verifiable	存在有限、可行、成本可接受的过程判定其是否被满足
Complete	陈述充分，无需再补充即可理解
Necessary	为达成目标所必需，删去即产生缺口
Feasible	在约束（技术、成本、进度）下可实现
Correct	准确陈述真实需求
Conforming	符合规定的模板与格式约定
Implementation-free	不预设实现方案（约束性需求除外）

INCOSE 《Guide to Writing Requirements》明确其特性表与 ISO/IEC/IEEE 29148、SEBoK、INCOSE Handbook 一致，可作为工程落地手册使用。

2.2 关键一条：Singular——"条目化"在标准里的名字

"The statement focuses on a single capability, characteristic, or constraint."

一条复合需求（含"和/或"的多能力陈述）**必须拆成多条——这不是风格偏好，是合格性判定**。所以"条目化"不需要另找理由：它是标准对"一条需求是否合格"的定义的一部分。

与它配合的还有 **Verifiable** 与 **Unambiguous**：只有单数、可验证、唯一解释的条目，才能被判定"是否被满足"——这个性质在 §四 会被再次用到（它决定了**什么才能被分配**）。

2.3 两个层面：单条需求 vs 需求集

29148 把特性分两层，必须分清：

- **单条需求**的特性 = 上表九条（Singular 属此层）。
- **需求集**的特性 = Complete（完整）、Consistent（一致）、Affordable（成本可承受）、Bounded（边界明确）等。

含义：条目化只解决“逐条合格”；“整体合格”是另一个层面的问题，不能靠把条目切细来获得（见 § 五）。

2.4 条目必须有身份

条目化不只是“拆开”，还要求每条**可被唯一标识与引用**。这是追溯 (§4.3) 的技术前提——没有稳定的条目身份，“跟随一条需求的生命”就无从谈起。

2.5 小结

条目化是强制性要求，其规范名字是 **Singular**；它由需求书写标准直接规定，并且天然带来两个副产品——**可验证**（每条能被单独判定）与**可追溯**（每条有身份可被引用）。

三、树形化：单元的组织

本问位置：条目（最小单位）之间如何组织成树。**依据领域**：系统层级理论（Simon）+ 分解纪律（WBS / MECE）+ 分类学与知识组织。**本问分两层**：先问“为什么要分层”（3.13.2），再问“**条目能否组装成树**”（3.33.5）——第二层才真正决定本问的答案。

3.1 分层是常态：Simon 的层级与近可分解性

Herbert Simon 在 *The Architecture of Complexity* (1962) 提出：复杂系统普遍表现为**层级（hierarchy）结构，且近可分解（nearly decomposable）**——子系统内部交互强、子系统之间交互弱。推论是：**可以分层设计与分层理解**，各层可以在相当程度上独立讨论。

这是“把产品数据组织成树、并逐层展开”的**元理论依据**。但请注意它的射程：它回答的是“**为什么可以一层层分下去**”，不回答“从条目能否组装出树”。

3.2 工程纪律：WBS 的 100% 规则与规格树

WBS（工作分解结构） 源于美国国防部标准 MIL-HDBK-881（后由 MIL-STD-881 取代），经 PMI 体系推广。其核心纪律是 **100% 规则**：

- 子层必须**完整覆盖**父层的全部范围（不遗漏）；
- 子层之间**互不重叠**（不重复）。

这与咨询业的 **MECE**（Mutually Exclusive, Collectively Exhaustive，相互独立、完全穷尽）是同一件事。把“工作”替换为“设计对象”，即得分解树必须满足的**完备性 + 互斥性**。

INCOSE 给出了它在工程中的形态：“**基于演进中的系统架构，产生一棵规格树（specification tree）**，用以定义层级化的表示...”——分层不是抽象比喻，工程实践中它就是一棵**树形的产品数据组织**，每层节点承载各自的条目集合。

树形化的关键产出是**末级节点（叶）**：它必须切到“可被独立处理”的粒度——可独立定义、可独立验证、可独立交给下一层。这与 §4 的向后分配互为条件：**叶子切不到可分配的粒度，分配就无法执行。**

3.3 “能组树”是平凡的：树 ⇔ 递归划分

在序理论中，“树”有精确的名字——**tree order（树序）**：一个偏序集，其中每个元素的前驱链是良序的。等价的说法是：**树 = 对一个集合做递归划分**——每个父节点的子节点，构成父节点所辖集合的一个不相交并。

由此得到一个必须先讲清楚的推论：**任何有限集合都能被组织成树**，只要给定任一划分并递归下去。

同时它也给出**树必有终止级（叶）**：对有限集合，任何划分链都必然终止，不可能无限细分下去。但要当心——“在哪一级停”是一个判断，不是数学结论：数学只保证“切得下去、也总切得完”，并不告诉你该停在哪。设计链上的停法见 §4.5。

所以真正的问题不是“能不能组”，而是“按什么判据组、组出来是不是树”。下面一节回答这个问题。

3.4 成树的三个条件

同一层的分组，必须同时满足三条，才得到一个合格的树：

条件一·判据单维。布利斯分类法把它写得最直白：

*“One fundamental rule is that division must be by **one principle only at a time**.”*——
例如给衣服分类，若同一次划分里既按功能又按颜色，就乱了。

一旦某层混用两个判据，同层内即出现**交叉分类（cross-classification）**，结果不再是树。而对象本质多维时，图书馆学并未硬掰，而是换了工具：**阮冈纳赞的分面分类（faceted classification）**——承认对象有多个面，放弃单一层级，改用“分面 + 引用次序”排序组合。**分面分类的出现本身，就是对“树不够用”的正式承认。**

条件二·条目单归属。在知识组织中，“每个条目恰好一个父节点”叫**单继承（single inheritance）**，对应**分类学（taxonomy）**；允许多个父节点叫**多层级（polyhierarchy）**，对应词表/本体，其结构是**有向无环图（DAG）**。医学词表（MeSH 一类）明确采用多层级结构，因为一个概念常同时属于多个上位类。

结论：说“条目能组树”，等价于说“这一层采用单继承”——这是一个要被声明的假设，不是被发现的属性。

一处需要说清的用词：上述术语与例证出自**分类学**——它分的是“**概念的种类**”（is-a：苏格拉底 is-a 人 is-a 动物）。而本报告关心的树分的是“**整体由哪些部分承担**”（整体—部分）。这是两种不同性质的层级关系；“一个节点只归一个父”对两者都成立，故规则可通用——但**不可**

把分类学的例证直接当成分解树的依据（“叶的终止判据”尤其不能这样借：它由可验证性给出，见 §4.5）。

条件三·分组完备互斥。 即 §3.2 的 100% 规则：不遗漏、不重复。

三种情形对照：

情形	结果
同层只用一个判据 + 每条条目只归一个父 + 分组完备且互斥	树 ✓
同层混用多个判据，或条目可归多个父	格 / DAG（多层级分类）——不是树
分组有遗漏或重叠	有缺口 / 重叠的树——不是合格的树

即：条目化的产品数据可以组装成树，当且仅当「**分组判据单维 + 条目单归属 + 分组完备互斥**」三条同时成立。这三条**不是自然成立的**——它们是设计约束，必须被显式声明并逐层检查。条件一尤其容易漏：一条“既按职责分、又按业务域分”的层级，看起来像树，实际是格。

3.5 另一条路：自底向上聚类及其局限

上述都是“从整体往外切”。反过来“从条目往上归”也有成熟方法，但代价不同：

- **层次聚类 / 树状图 (dendrogram)**：给定条目间的相似度后递归合并，产出树形结构（Sokal & Michener 的数值分类学传统）。前提是存在一个**可计算的判据**（相似度或属性）。
- **形式概念分析 (Formal Concept Analysis, Ganter & Wille)**：由“对象 × 属性”的形式背景形式化地导出概念层级。注意它导出的是**概念格 (lattice)**，一般**不是树**——这从反面印证了 §3.4：层级化到什么程度，完全由判据决定。

3.6 小结：三个条件，以及两种成树方式

成树的三个条件：判据单维 + 条目单归属 + 分组完备互斥（§3.4）。

成树的两个方向，判据性质与权威来源都不同：

方向	机制	判据性质	代表方法
自顶向下分解 (decomposition)	从整体往外切	设计意图 ，必须可辩护	WBS、公理化设计
自底向上聚类 (clustering)	从条目往上归	相似度/属性 ，必须可计算	层次聚类、形式概念分析

设计链上的树，来源通常是前者——**条目不是原料，是挂点**。把两者混起来，就会出现“树很整齐，但没人能说清它为什么这么分”。

四、向后分配：层间的连接

本问位置：两份产品数据之间如何连接。**依据领域：**系统生命周期标准（15288）+ 设计理论（公理化设计）+ 模块化理论（Parnas）。**本问分两层：**先问“分配为什么发生”（4.14.3），再问“分配为什么是这个形状”（4.44.7）——即为什么是“叶 → 唯一枝梢、基数 N:1”。第二层理论依据最厚，也最容易被“看来自相矛盾”质疑（见 4.7）。

4.1 过程依据：标准把“上一层产物”定义为“下一层的需求”

ISO/IEC/IEEE 15288（系统生命周期过程）的技术过程序列是：

```
Business or Mission Analysis
  → Stakeholder Needs and Requirements Definition
  → System Requirements Definition
  → Architecture Definition
  → Design Definition
  → System Analysis → Implementation → Integration → Verification → ...
```

这条链与“相关方需求 → 系统需求 → 架构/构件设计”逐段对应。**关键在于：**这些过程的输出统统被命名为 **requirements**，即上一过程的产物就是下一过程的输入需求。

SEBoK 讲得更直白：

*“The logical architecture defines system boundary and functions, **from which more detailed system requirements can be derived.**”*

含义：方案侧的架构会反向“导出”下一层的需求条目。所以链条的每一环天生都是“条目集合”——这是“每层都可条目化”的过程依据，而不是一种组织偏好。

4.2 方法论依据：公理化设计（Axiomatic Design）

Nam Pyo Suh（MIT）的公理化设计是最直接的理论依据。它把设计拆成两个域：

概念	定义
FR（Functional Requirement，功能需求）	表征设计目标的 最小独立需求集
DP（Design Parameter，设计参数）	物理域中为满足 FR 而选定的关键变量

二者以**设计矩阵**关联： $\{\text{FR}\} = [\text{A}]\{\text{DP}\}$

两条公理：

- **独立性公理（Independence Axiom）**：*maintain the independence of functional requirements*。要求设计矩阵为**对角**（非耦合，每 FR 由各自的 DP 独立满足）或**三角**（解耦，需按序调节）；否则即**耦合设计**（coupled design），不可接受。

- **信息公理 (Information Axiom)**：在设计满足独立性公理的候选中，选**信息含量最小者**。

zigzag 分解： $FR_1 \rightarrow DP_1 \rightarrow FR_2 \rightarrow DP_2 \rightarrow \dots$ 逐层交替展开——这正是“需求→方案→需求→方案”逐层推进的一般化理论表述。而独立性公理给出了“条目为什么要切到彼此独立”的理论理由：**独立性不足的需求集无法被独立地分配与验证**。

4.3 追溯性：条目化与分配是可追溯的前提

Gotel & Finkelstein (1994) 对可追溯性的经典定义是：

描述并跟随一条需求**生命**的能力，包括*前向（需求规格之前）与后向（需求规格之后）两个方向。*

要“跟随一条需求的生命”，这条需求必须是有**标识、可引用的个体**，且必须知道它被**分配到了哪里**。因此：

条目化 (§二) 与向后分配 (§四) 不是各自独立的做法，而是可追溯性的两个必要前提。

4.4 分配的落地形式

- **公理化设计**把设计定义为四个域之间的映射： $customer (CN) \rightarrow functional (FR) \rightarrow physical (DP) \rightarrow process (PV)$ ，并以 $\{FR\}=[A]\{DP\}$ 刻画功能域到物理域的映射。“设计”在这里就是映射本身，没有别的成分。
- **ISO/IEC/IEEE 15288** 用工程语言说同一件事：标准中明确 “*requirements are allocated to system elements*”；**SysML** 更直接——它有一个专门的 **Allocation** 关系，用来把需求关联到结构/行为元素。

含义：一旦把需求与方案分成两个域，连接它们的**唯一动作就是分配**。所以“设计过程落地为分配”是一个**建模选择**，不是巧合。

4.5 定义域：为什么是前序的“叶”

- **ISO/IEC/IEEE 29148** 要求单条需求 **Singular + Verifiable (§2.2)**：只有单数、可验证的条目，才能被判定“是否被满足”。
- 粗粒度节点（簇、枝梢）**不能被 1:1 分配**——它内部含着多条性质不同的需求，交出去后无法判定谁负责哪一部分。
- 因此分配的操作对象必须是**已原子化的条目**；在需求侧，原子就是**叶**。而“在哪一级停”是个判断 (§3.3) ——本报告采用的判据是**可验证性**：叶必须切到**能被独立判定“是否被满足”的粒度**。它天然是该侧最小的可交接单位。

4.6 值域：为什么是后续的“唯一枝梢”

这里有一个关键点，容易误解：

"叶 → 枝梢"不是跨了两级，而是两侧"各自最深可用原子"的对齐。

- 分配的目标必须是**能负责的实体**——标准里的对应词是 **system element**（可被当作一个整体处理的系统成员）。
- 在这条链的这一刻，**方案的叶还不存在**：方案只产出了概要（骨架），它的叶要等下一阶段分解出来。
- 所以方案侧能提供的最深实体，就是它的**架构末级节点 = 枝梢**。

即：**需求侧已到叶（它的完整态已出），方案侧只到枝梢（它的完整态待出）**——"错开一级"反映的是**两侧完成度不同步**，不是层级本身有差。

为什么不是"需求叶 → 方案叶"：方案的叶是"该实体怎么实现"的细节，其来源是该枝梢往下的分解（属同一份方案数据内部下一阶段的工作），不是上游需求的直接输入。**上游需求只规定"这个实体要满足什么"，不规定"这个实体怎么拆"**。

4.7 基数：允 N:1、禁 1:N——两者理由不同

这一点最需要拆开，因为很多人默认二者是对称的。

(a) N:1 允许 ← 模块内聚。 一个方案实体天然可以承担多项需求。**Parnas 的信息隐藏原理**说：模块的边界按"要隐藏的设计决定"来划，一个模块封装一族决定，因此对应**多条需求**。ISO 15288 的分配同样允许一个系统元素承接多条需求——**该元素的需求集合就是它的规格**。这是期望形态，不是缺陷。

(b) 1:N 禁止 ← 变更传播 + 重复实现。

- 一条需求若由多个实体共同满足，则该需求一旦变更，**多个实体都要改**——变更无法局部化。这正是模块化理论衡量耦合的核心指标（Parnas; Baldwin & Clark 的模块化设计规则）。
- 同时带来**重复实现**风险：同一需求被实现两遍甚至不一致，追溯上也说不清谁真正负责。
- **公理化设计**给出同一结论的另一种表述：一条 FR 若需两个 DP 共同满足，两个 DP 就通过该 FR **耦合**，设计矩阵非对角也非三角 → 耦合设计。

(c) 必须承认的不对称。 在严格的公理化设计里，**N:1 与 1:N 同为非对角、同属耦合**。这条规则允许前者、禁止后者，说明它用的判据**不是纯粹的独立性公理**，而是"**需求侧独立 + 方案侧内聚**"的工程平衡：

	需求侧（独立）	方案侧（内聚）
1:N （一条需求 → 多个实体）	× 需求被拆散，无法单独验证	—
N:1 （多条需求 → 一个实体）	可接受	✓ 内聚实体天然承接一族需求

只有当同一实体内多条需求对应的实现相互影响时，N:1 才重新变成问题。——这恰是"模块内部是否需要保持独立"的问题，不是分配规则本身能解决的。

(d) **实践含义**。分配的基数 (1:1 / N:1 / 1:N) 不是一个可以默认的选择，而是**必须显式定义并给出理由**的设计决策——因为不同基数对应不同的设计耦合度。一般标准 (15288 / INCOSE) 要求分配发生且双向可追溯，但**不强制规定基数**；公理化设计也只给出偏好 (对角/三角)，不给出禁令。

4.8 小结

设计过程可以落地为「叶 → 唯一枝梢、基数 N:1」的分配，因为设计被建模为从需求域到方案域的映射；映射的定义域取需求侧的原子（叶），值域取方案侧当前可负责的实体（枝梢）；而映射必须是函数——函数允许多对一（内聚），不允许一对多（耦合）。

这条规则隐含三个前提，**都可检查**：

1. 需求侧**已完成原子化**（叶存在）——否则没有可分配的单位；
2. 方案侧**已定义出可负责的实体**（枝梢存在）——否则没有可承接的对象；
3. 两侧的原子**粒度可比**——否则出现“粒度不匹配”（29148/INCOSE 的典型反模式：“一条需求怎么都落不到唯一的实体上”）。

五、限度：这些结论的适用边界

前面三章讲的是“成立”，本章讲“限度”。外部文献自己划了边界：

1. **耦合设计普遍存在**。公理化设计本身承认：真实系统中独立性公理常无法完全满足，设计矩阵出现非对角项是常态。此时要么**解耦**（引入调节顺序）、要么**显式接受耦合**——而不是假装它已被切开。
2. **需求集的完整（Complete）难以形式化证明**。29148 的 Singular 只作用于单条需求 (§2.3)；“整体不漏”只能靠评审与追溯覆盖来**主张**，不能靠条目切细来**保证**。
3. **因此，“能否条目化”本身是一个判据，不是一个断言**。可操作的判据是：

该条目**能否被独立解释、独立验证，并能否被唯一地交出去**。

切不动的，应走三个出口之一：**合并**（承认它本是一条）、**降级为约束**（不作为可分配条目）、或**显式标注耦合**（记录其与哪些条目共享实现）。

六、对本项目（EOS 设计线）的意味

本节属应用推论，不作为 §一~§五 的论据。

上述三条主线分别对应产品数据文档应当具备的三种属性：

- **条目化** ↔ **"叶节点只在完整态产出"**：叶 = 可独立验证、可交出去的条目，对应 Singular + Verifiable 的落地形态。
- **树形化** ↔ **"概要态（根→枝梢） / 完整态（根→叶）"**：概要是可独立讨论的骨架（层级 + 100% 规则的组织纪律），完整态补上可分配的末级条目。§3.4 的三条件在此可直接落地为三条检查：**该层是否只用一个判据**（如"按系统职责"而非职责与业务域混用）、**一个条目是否只归一父**（单归属 vs 多归属）、**分组是否完备互斥**。
- **向后分配** ↔ **"上一份的叶节点由下一份的枝梢节点承接"**：§4.4~4.8 给出这条形状的完整理由——定义域取需求侧原子（叶）、值域取方案侧当前可负责实体（枝梢）、映射必须是函数（允 N:1、禁 1:N）。**其最关键的一条是 §4.7(c) 的不对称**：N:1 与 1:N 不是镜像，N:1 的合法性来自**模块内聚**（Parnas）、1:N 的非法性来自**独立性/变更传播**（Suh + Parnas）——**当前项目对基数只写了形状（允 N:1、禁 1:N），没写这条不对称的来源**，所以看起来像自相矛盾（"耦合既是坏的，N:1 为何是好？"）。

三处值得复核的接口：

1. §4.7(d) 指出基数应当"显式定义并给出理由"、且 **N:1 与 1:N 的理由不同源**。当前项目对基数的表述是结论式的；按外部依据，它还欠一段"为什么"，并且这段"为什么"**应当把两个方向分开写**。
2. §3.4 条件二指出**树成立的前提是单继承**。若某一层的节点存在多归属（同一节点同时属于多条树），那它就不是树——这一点需要在树的定义处显式排除或声明。
3. §4.6 指出**"叶 → 枝梢"不是跨级，而是两侧"各自最深可用原子"的对齐**（错开一级 = 完成度不同步）。若把这条读成"上游的叶对应下游比它低一级的东西"，就会推出错误的层级换算规则。

落点（2026-09-10）：以上三条已由 94 于 v1.5 / v1.6 落定——94 §1.2「**本文各树凭什么成立：三个前提**」（三问三答：为什么可以条目化 / 可以树形化 / 可以 N:1 分配，对应本条 2 与 1 的判据面）与 §1.4 **反误读注**（对应本条 3）。**边界**：94 只给结论与依据名称，完整论证仍以本报告为准。

七、来源

核查说明：以下链接为本报告的依据出处。受本机网络策略限制，正文未能逐字抓取，故引用以摘要与标准通行内容为准；**建议以标准原文复核关键措辞**。分组按依据领域，与正文章节对应。

标准与手册（→ §二、§三、§四）

- ISO/IEC/IEEE 29148:2018 — *Systems and software engineering — Life cycle processes — Requirements engineering* (良构需求特性清单)
- ISO/IEC/IEEE 15288:2015 — *Systems and software engineering — System life cycle processes* (技术过程链)
- MIL-HDBK-881 / MIL-STD-881 — *Work Breakdown Structures for Defense Materiel Items* (100% 规则) · [标准页](#)
- [INCOSE Guide to Writing Requirements v4](#)
- [INCOSE Systems Engineering Handbook](#) (技术过程)
- [PPI — Review of ISO/IEC/IEEE 15288:2015](#)
- [SEBoK — System Requirements Definition](#) · [SEBoK v2.5 全文 PDF](#)

29148 特性的二次转引 (→ §二)

- [九条个体需求特性名录](#) (microsoft/hve-core 引用文档)
- [Singular 定义转引](#) (TUM 学位论文)
- [Unambiguous 定义](#) (TU/e 软件规格说明教材)
- [面向 29148-2018 的良构需求 NLP 校验框架](#)

层级、分解与成树的条件 (→ §三)

- Herbert A. Simon, *The Architecture of Complexity*, 1962 · [层级理论综述](#) · [Principle of Recursive Systems Construction](#)
- [MECE 原则与 WBS 技巧](#)
- [Bliss Classification, Introduction](#) ("division must be by one principle only at a time")
- [Ranganathan and the faceted classification theory](#) (分面分类) · [Colon Classification](#) (Britannica)
- [NLM — 层级关系与多层级 \(polyhierarchy\) 结构](#) · [多层级分类构建方法](#) (CA2536179A1)
- [Ganter & Wille, Formal Concept Analysis](#) (概念格, Zbl 0861.06001)
- [层次聚类 / 链接法](#) (WPGMA)
- [Parnas, 模块 uses hierarchy](#)

公理化设计 (Suh) (→ §四)

- [MIT — Axiomatic Design 导论](#)
- [MIT 2.882 — 设计公理](#)
- [ScienceDirect — Independence Axiom](#)
- [Axiomatic Design 与设计矩阵](#) (硕士论文)
- [Elements of Axiomatic Design \(Suh\)](#) · [A Deep Dive into Axiomatic Design, Part I](#)

分配 (allocation) 与模块化 (→ §四)

- [SEBoK — System Architecture Design Definition](#) · [SEBoK — Physical Architecture](#)
- [SysML Allocation 关系](#)
- [VTT — Requirements Allocation](#) ("Each system level requirement ... is allocated")
- [ScienceDirect — Requirements Allocation](#) · [Requirement Allocation in Systems Engineering Explained](#)
- [Parnas 的信息隐藏判据](#) · [Baldwin & Clark 模块化](#) · [CMU/SEI — 需求 flowdown 与分配错误](#)

可追溯性 (→ §4.3)

- [Gotel & Finkelstein \(1994\) 定义转引](#) · [A scoped approach to traceability management](#) · [需求追溯实践指南 \(SodiusWillert\)](#)

变更记录

日期	版本	说明
2026-09-10	v0.5	删去 §3.5 「叶从何而来：属加种差」，修正一处“不承担论证功能”的引用（人类指出该节与本报告目的一致性问题）——原 §3.5 有三处不成立：① 归属错位（“叶”是 §2 条目化的产物，§4.5 已明言“在需求侧，原子就是叶”，§3.5 在 §3 里对同一问题另起一套解释）；② 判据冲突（§3.5 给的是概念可定义性判据“按同一判据再也分不下去”，而报告实际用的是可验证性/可分配性判据——§2 Verifiable + §4.5；§4.5 原句“与 §3.5 的古典依据吻合”是把两套判据说成一套）；③ 层级性质未辨（波菲利之树是 is-a 分类 装置，分的是概念的种类；报告的树是整体—部分分解）。且该节结论全文仅在 §4.5 出现一次呼应式引用，删去后 §3.4 与 §4.5 均不缺内容。处理：① §3.3 末补“树必有终止级（叶）”及关键限定——“在哪一级停是一个判断，不是数学结论”（数学只保证切得完，不告诉你停在哪）；② “分类 vs 分解”的区分补进 §3.4 条件二（那里正在使用分类学的词与例证，正是该说清之处，并说明“规则可通用，但不可把分类学的例证当成分解树的依据”）；③ §4.5 改引 §3.3 并直接陈述判据（可验证性）；④ §3.6/§3.7 顺次上移为 §3.5/§3.6，章内引语与来源清单同步（删去 Porphyry 来源条）。
2026-09-10	v0.4	通篇重构（布局 + 表述），目标：更有逻辑性、更便于阅读——① 头部新增摘要与术语表（条目/叶/枝梢/分配/需求侧·方案侧/FR·DP/system element 集中定义）；② §一 改为「问题与结论」，纳入原 §六 的“三问关系”（单元 → 组织 → 连接）并新增 §1.4 「各章的展开方式」（声明三章统一按“依据 → 条件/形状 → 小结”展开）；原 §六 删除（与 §一 重复）；③ 三章标题统一为“对象 + 问题”：条目化 = 产品数据的最小单元 / 树形化 = 单元的组织 / 向后分配 = 层间的连接；每章开头加统一引语块（本问位置 + 依据领域 + 分层说明）；④ §三 内部重排：先“分层是常态”（3.13.2, 原 3.13.3 合并），再“能组树是平凡的”（3.3, 原 3.4），再“成树的三个条件”（3.4, 把原 3.6 判据单维 + 3.7 单继承 + 3.2 完备互斥合并为三条并列条件，附三情形对照表），再“叶从何而来”（3.5, 原 3.5），再“另一条路”（3.6, 原 3.8），末尾 3.7 小结；⑤ §四 增设 4.8 小结；⑥ 限度从

日期	版本	说明
		各章收拢到 §五 (统一"反面边界", 避免与各章小结重复); ⑦ §七 来源按依据领域分组并标注对应章节; ⑧ 原 §七「对本项目的意味」顺次为 §六。
2026-09-10	v0.3	§四 补第二层论证: 分配为什么是这个形状——原 §四 只答"分配为什么发生" (15288 过程链 / 公理化设计 / 追溯性), 未答"为什么是叶 → 唯一枝梢、N:1"。新增 4.4~4.8: 4.4 设计的落地形式就是分配 (AD 四域映射 + 15288 "requirements are allocated to system elements" + SysML Allocation); 4.5 定义域 = 前序的叶 (29148 Singular/Verifiable 决定只有原子条目可被分配); 4.6 值域 = 后续的唯一枝梢 ("叶 → 枝梢"不是跨级, 而是两侧"各自最深可用原子"的对齐——需求侧完整态已出、方案侧只到概要; 并答"为什么不是需求叶 → 方案叶"); 4.7 基数 = 允 N:1、禁 1:N, 且两侧理由不同源 (N:1 ← Parnas 信息隐藏/模块内聚; 1:N 禁止 ← 变更传播 + 重复实现 + AD 耦合), 并点明不对称: AD 里 N:1 与 1:N 同为耦合, 故本规则的判据是"需求侧独立 + 方案侧内聚"的工程平衡; 4.8 结论与三个隐含前提。§一 结论表"向后分配"行同步; §七 三条复核接口重写 (第 1 条改为"N:1/1:N 的理由不同源、项目只写了形状"); §八 新增"分配 (allocation) 与模块化"来源组。
2026-09-10	v0.2	§三 补第二层论证: 从条目集到树的条件——原 §三 只答"为什么分层" (Simon / WBS / 规格树), 未答"条目能否组装成树"。新增 3.4~3.9: 3.4 数学层 (树序 ⇔ 递归划分, 故"能组树"平凡成立, 真问题在判据); 3.5 古典依据 (波菲利之树: 属加种差, 并给出"叶 = 按同一判据分不下去的那一级"); 3.6 关键条件 (Bliss 分类法"division must be by one principle only at a time"——同层混用判据即交叉分类, 不再是树; 分面分类是对"树不够用"的正式承认); 3.7 树 = 单继承的显式假设 (多归属只能得 DAG); 3.8 自底向上造树 (层次聚类 dendrogram / 形式概念分析得概念格而非树); 3.9 结论 (成树三条件: 判据单维 + 条目单归属 + 分组完备互斥; 且区分自顶向下分解 vs 自底向上聚类, 权威来源不同)。§一 结论表"树形化"行与"本报告节"列同步 (后者顺带修正初稿整体串一位的错: 条目化 §三→§二、树形化 §四→§三、向后分配 §五→§四); §2.2/§4.4 内引同步修正; §七 补三条复核接口; §八 新增"成树的条件: 分类学与知识组织"来源组。
2026-09-10	v0.1	初稿——就"产品数据能否条目化 / 树形化 / 向后分配"作外部理论梳理: 条目化 ← ISO/IEC/IEEE 29148 良构需求特性 (Singular / Unambiguous / Verifiable); 树形化 ← Simon 层级与近可分解性 + WBS 100% 规则 + INCOSE 规格树; 向后分配 ← ISO/IEC/IEEE 15288 技术过程链 + SEBoK「架构导出更细需求」+ 公理化设计 FR↔DP zigzag 与独立性公理 + 追溯性定义。含边界与反例 (耦合设计、需求集完整不可形式化证明、"能否条目化"是判据而非断言) 与来源清单。